Using Weisfeiler-Leman Features for Algorithm Selection in Constraint Optimisation

Anonymus
Anonymus

Abstract

We have new features and they are decent enough.

2012 ACM Subject Classification Computing methodologies → Artificial intelligence

Keywords and phrases Constraint modelling, algorithm selection, machine learning, constraint solvers.

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

2 Background

3 Graph Conversion

Although seeing a combinatorial problem as a graph is quite natural, how to structure it is less obvious. Over the years, many different approaches have emerged: many MIP-related tasks use a bipartite graph representation where a pair of variable/constraint nodes share an edge when the variable has a non-zero coefficient in the constraint [3]. Another possibility, often used in CP, is to represent the problem as a tripartite graph, where nodes are constraints, variables and possible values that variables can take. Similarly to the bipartite version, edges are present between variables and constraint nodes if a variable has a non-zero coefficient in the constraint. Furthermore, variable nodes also have edges that connect them to all possible values they can take [2]. Finally, Boisvert et al. [1] decide to decompose constraints into smaller components, and the final graph is akin to the abstract syntax tree of a program.

Our approach is inspired by that of Boisvert et al. [1], but, unlike them, we do not include domain values in our representation; furthermore, our graphs are directed instead of undirected. We have decided to use Boisvert et al.'s syntax tree-like representation as it allows us to decompose complex constraints into simpler elements that could be reused in different constraints with similar meanings.

In more details, we define five types of nodes:

- **Variable nodes**: representing the variables of the problem.
- **Solution node**: a single node for each problem that represents the type of problem (Minimisation, Maximisation or Satisfaction).
- **Parameter/Literal nodes**: representing the literal values of the problem (e.g. 1, 2, True, False, etc.).
- **Operator nodes**: representing arithmetic and functional operations between variables, parameters and other operators (e.g. sum, multiplication, division, etc.) Operator nodes have a sub-category for the type of operator represented.
- **Constraint nodes**:representing relational predicates (e.g. $=$, $\neq$, $<$, $\leq$), logical operations (e.g. $\wedge$, $\vee$, $\neg$), and global constraints (e.g. `all_different`, `table`). Constraint nodes also have a sub-category for the type of constraint the node represents.

All node and sub-node types we have defined can be seen in the appendix.

As previously mentioned, edges between nodes are directed: variables and parameter nodes have no incoming nodes and only point to the operators or constraints they take part in. There is only one exception: for optimisation problems, the variable containing the objective value to optimise has an

incoming edge from the Solution node. Operator nodes can have both incoming and outgoing edges. Incoming edges come from variables, parameters and other operators that take part in the operation. Outgoing edges go to other operator nodes or to constraint nodes. Constraint nodes can also have edges to other constraint nodes when they appear as a sub-expression to other constraints. As an example, the expression $(x = 3) \wedge (y \neq 5)$, will have three constraint nodes ($=$, $\wedge$ and $\neq$) with $=$ and $\neq$ having exiting edges to $\wedge$.

Edges also have types; in particular, there are three types of edges:

- **Edge of type 0**: used to connect variables, parameters and operators to operators and constraints for which the order of the operands does not matter (e.g. sum or equality). 0 Edges are also used for variables, parameters and operators that fall on the left side of an operation or constraint where the order of the operands matters (e.g. subtraction or strictly less than). Type-0 edges are also used to connect the solution node to the objective variable in optimisation problems.
- **Edge of type 1**: Used to connect variables, parameters and operators to operators and constraints for which the order of the operands matters and the operands fall on the right side of the operation.
- **Edges of type 2**: Used to connect variables and parameters to global constraints. This helps to distinguish normal connections from normal and global constraints.

```
var int x;
var int y;
constraint 3*x <= y;
constraint y < 10;
all_different([x, y]);
solve maximise x;
```

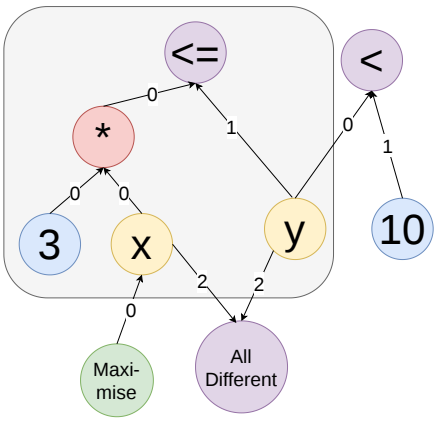

Figure 1 Conversion of a constraint model (left) to a graph (right). Variables in yellow, Parameters in blue, Operators in red, Constraints in purple, and Solve in green. The area in gray represents the constraint `3*x <= y`.

Figure 1 shows an example of how a simple constraint model can be converted using our conversion methodology. In many previous works, like [1], it has been argued that adding domain values to the graphs was essential to get a proper representation. The core of the argument was that, without including such elements, the conversion function would not be injective and, as a consequence, two different problems could fall into the same representation. While this argument is correct, adding such information adds a lot of nodes to the graph, and it could make the conversion too expensive for problems with big variable domains. Our primary objective is to use the converted graphs to extract features for algorithm selection purposes, and, as an example, an instance from the 2025 Minizinc challenge[1] (coming from the cgt problem with instance name cgt_12_h27_r0.05_s0.5_3) had an objective function with a domain between -1.932.060 and 1.714.644, resulting in 3.646.705 total possible values just for one domain. The full instance would also require adding also other

[1] https://www.minizinc.org/challenge/2025/results/

Algorithm 1 Standard WL Algorithm for Graph Isomorphism

Require: Graphs $G = (V_G, E_G)$ and $H = (V_H, E_H)$
Ensure: *True* if $G \simeq H$ (potentially), *False* otherwise
$c_G^{(0)}(v) \leftarrow 1$ for all $v \in V_G$, $c_H^{(0)}(u) \leftarrow 1$ for all $u \in V_H$
$t \leftarrow 0$
repeat
  $t \leftarrow t + 1$
  for each node $v \in V_G, u \in V_H$ **do**
    $M_G(v) \leftarrow \{\!\{c_G^{(t-1)}(w) \mid (v, w) \in E_G\}\!\}$
    $M_H(u) \leftarrow \{\!\{c_H^{(t-1)}(w) \mid (u, w) \in E_H\}\!\}$
    $c_G^{(t)}(v) \leftarrow \text{HASH}(c_G^{(t-1)}(v), \text{sort}(M_G(v)))$
    $c_H^{(t)}(u) \leftarrow \text{HASH}(c_H^{(t-1)}(u), \text{sort}(M_H(u)))$
  end for
  if $\{\!\{c_G^{(t)}(v) \mid v \in V_G\}\!\} \neq \{\!\{c_H^{(t)}(u) \mid u \in V_H\}\!\}$ **then**
    return *False*
  end if
until partitions induced by $c_G^{(t)}, c_H^{(t)}$ are stable
return *True*

variables and their domains as well as constraints and operators. A graph with this many nodes would be intractable for our purposes.

4 Extracting Features Using The Weisfeiler-Leman Algorithm

Once we have defined how to convert any instance to a graph, we can use the Weisfeiler-Leman (WL) algorithm to extract features from the graphs.

4.1 The WL algorithm

The original purpose of the WL algorithm is to check if two graphs are isomorphic [4]. It does so by assigning a colour to each node on a graph and then iteratively refining it by aggregating its colour with the colours of the neighbouring nodes. The algorithm would keep refining the colours until either (a) the two graphs have a different colour set or (b) the refinement did not change the induced partitioning of the graphs.

It is important to note that the WL algorithm does not guarantee correctness, and two graphs can be deemed isomorphic even though they are not. Typically, this happens when the two input graphs have the same node degree while still having a different structure.

It is also possible to generalise the Wl algorithm to the k-WL algorithm where the colours are assigned to a k-tuple of nodes. In this sense, the standard WL algorithm is a special case of the k-WL algorithm where $k = 1$. At the growth of k, the k-WL algorithm gets much more powerful in distinguishing non-isomorphic graphs; however, its complexity also grows exponentially in k. Thus, for the purpose of this paper, we will only focus on the standard WL algorithm. The implementation of the WL algorithm for graph isomorphism can be seen in Algorithm 1.

4.2 WL For Feature Extraction

To use the WL algorithm for feature extraction, we repeat the colour refinement process for a pre-defined number of steps on a single graph, then we extract features by creating a histogram where, for

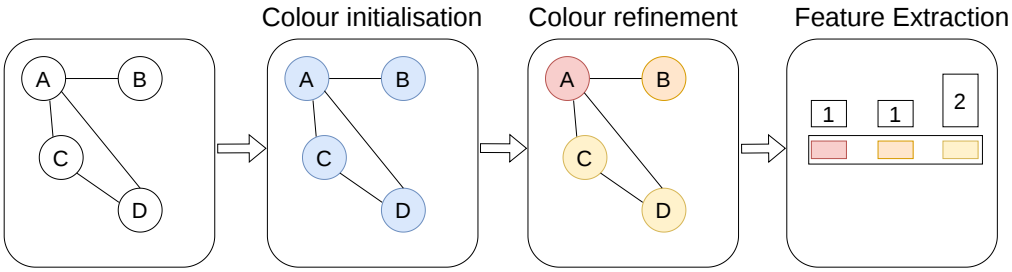

Figure 2 Feature extraction process using the WL algorithm on a simple, undirected graph. The colour refinement is executed only once. At first, all nodes get the same colour, then node A aggregates its colour (blue) with its 3 neighbours' colours (blue), getting, as a result, (blue, blue,blue,blue). This produces a new colour, red. C and D, having only two neighbours, get, as aggregation (blue, blue,blue), resulting in the new colour yellow and, finally, node B, having only one neighbour, aggregates to (blue, blue), getting as new colour orange. In the final steps, we count the number of occurrences of each colour, and we get 1 red, 1 orange and 2 yellow.

each colour, we count its occurrences in the graph. A depiction of the execution of the WL algorithm for feature extraction (with one aggregation step) can be seen in Figure 2. The WL features have been proven effective in many fields such as cheminformatics and computer vision. For a comprehensive review of the WL features for ML, we will refer to Morris et al. [5].

4.3 Enhancing Features' Expressiveness

The standard WL features are designed for graphs without any edge or node information. However, in our case, we have both node and edge-level information we can use to better represent the input graph. For this reason, we propose two extensions to the base algorithm:

- **Node-information extension**: To add node information into the extraction process, we can initialise the node colour with the hash of its type + subtype (e.g. a `Variable node` and a `Sum node` will be initialised with different colours). Formally, $c^{(0)}(v) \leftarrow \text{HASH}(\text{type}(v) \,||\, \text{subtype}(v))$ where $||$ denotes string concatenation.
- **Edge-information extension**: To add edge information into the extraction process, we can concatenate the edge type to the node colour during the aggregation step (e.g. in the example of Figure 2, if Node B was connected to node A via a type-0 edge, the aggregation of node B would be: (blue, blue-0)). Formally, assuming a directed graph where edges flow towards v, its multiset update becomes $M(v) \leftarrow \{\!\{c^{(t-1)}(w) \,||\, \text{type}(w,v) \mid (w,v) \in E\}\!\}$.

Finally, we can use the fact that our graphs are directed and only update a node colour based on its incoming neighbours.

Using these extensions, we can now define four types of features: (i) `WL`, standard WL features without node or edge information, (ii) `WLn`, WL features enhanced via node-information extensions, (iii) `WLe`, WL features enhanced via edge-information extensions, and (iv) `WLne`, WL features enhanced with both edge and node-information.

4.4 The "Cut" Trick

In the WL algorithm, the colour of a node is highly dependent on its degree, and two nodes with different degrees, even when sharing the same original colour and the same type of neighbour nodes, will end up with different colours. This can be problematic for nodes representing operators and constraints where the number of connected sub-elements can vary. As an example, let's examine the `all_different` global constraint: using the standard algorithm, `all_different([v1, v2,`

`v3])` will get a totally different and uncorrelated colour to `all_different([v1, v2])` solely due to its degree. In practice, while preserving some differences between the two cases, we would still capture the fact that the two constraints are pretty similar.

To achieve this, we can virtually “cut" the connections between the nodes representing constraints and operators with a variable number of operands in them. We will refer to these nodes as cut nodes. In practice, during the aggregation steps, the cut nodes will not be updated since their incoming connections will be ignored. However, the cut nodes will still contribute to the update of the nodes they are connected to via their outgoing edges. All cut nodes can be seen in the appendix.

After having done so, and after the aggregation steps have been completed, each cut node’s colour can be updated by aggregating it with the set (not multiset) comprised of all its connected, original, node colours. By using a set rather than a multiset, nodes with different degrees but the same types of incoming neighbours will receive identical colours.

At this stage, we can correctly capture the fact that two constraints with the same operator, incoming edges but different degrees, have the same colour; however, we are completely missing any information regarding the degree of a node, information that we would still like to capture. To do so, we can add extra features after the colour histogram. Specifically, from each cut node, we can generate node pairs comprising the cut node and its incoming neighbours. Thus, each cut of degree n will generate n pairs. After having generated all the pairs, we can count the occurrence of each pair in a graph to generate a histogram of pairs and concatenate it to the histogram of colours.

The cut trick can only be applied when node information is present, thus, only to `WLn` and `WLne` features. We will call the features where the cut trick is applied `WLc` if only the cut is applied to `WLn` features and `WLce` if the cut is applied to `WLne` features. A formalisation of the `WLc` features can be seen in Algorithm 2, and a graphical depiction of how the cut trick is applied can be seen in Figure 3.

Algorithm 2 `WLc` Algorithm for Feature Extraction

Require: Graph $G = (V, E)$, iterations T, subset $V_{cut} \subseteq V$ of cut nodes
Ensure: Feature histograms of colours and pairs

Initialisation: $c^{(0)}(v) \leftarrow \text{HASH}(\text{type}(v) \,||\, \text{subtype}(v))$ for all $v \in V$
$t \leftarrow 0$
repeat
  $t \leftarrow t + 1$
  for each node $v \in V$ **do**
    if $v \notin V_{cut}$ **then**
      $M(v) \leftarrow \{\!\{c^{(t-1)}(w) \mid (w, v) \in E\}\!\}$
      $c^{(t)}(v) \leftarrow \text{HASH}(c^{(t-1)}(v), \text{sort}(M(v)))$
    end if
  end for
until $t = T$
$Pairs \leftarrow \{\!\{\}\!\}$ ▷ Initialize multiset for pairs histogram
for each node $v \in V_{cut}$ **do**
  $S(v) \leftarrow \{c^{(0)}(w) \mid (w, v) \in E\}$ ▷ Set of initial colours (types) of incoming neighbours
  $c^{(final)}(v) \leftarrow \text{HASH}(c^{(T)}(v), \text{sort}(S(v)))$
  for each node w where $(w, v) \in E$ **do**
    $Pairs \leftarrow Pairs \cup \{\!\{(c^{(0)}(w), c^{(0)}(v))\}\!\}$
  end for
end for
$c^{(final)}(v) \leftarrow c^{(T)}(v)$ for all $v \in V \setminus V_{cut}$
return $\text{Histogram}(c^{(final)}) \cup \text{Histogram}(Pairs)$

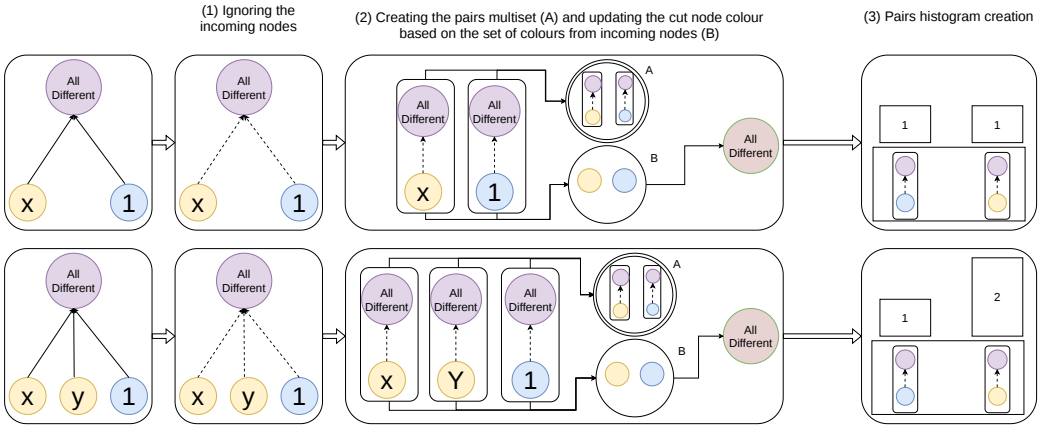

Figure 3 Application of the cut trick on the `all_different` constraint. Constraint nodes are in purple, variable nodes in yellow and parameter nodes in blue. First, we ignore colours from nodes with incoming edges (1), then we generate all pairs and update the colour of the `all_different` node by aggregating it with the set of all pairs (2) and then we generate the pairs histogram by counting all the occurrences of the pairs (3).

4.5 Features, Characteristics, and Normalisation

As a consequence of our design choices, especially of having directed graphs, the extracted features have emerging properties that are worth noting. In particular, since variable and parameter edges have no incoming node (but for the variable connected to the objective function), the colours of those values never get updated. Thus, one value in the final feature set would represent the number of variables in the graph, and another the number of parameters in the graph.

To give some extra information about variables and parameters in the graph, besides their total count, we decided to add two extra features, one representing the average number of constraints/operators a variable takes part in and one representing the same but for parameters.

For similar reasons, most paths in the graph are quite shallow, especially after the cut. As an example, in the `x + 3 = y` constraint, we will have 2 variable nodes, (`x, y`), one parameter node (`3`), one operator node (`+`) and one constraint node (`=`). The operator node can be updated only once since the values of the parameter and variable nodes connected to it will never get updated. At the same time, the value of the constraint node can be updated at most twice: once at the first step when it aggregates with the original `+` and `y` colours and once at the second step when it aggregates with the updated `+` colour and the original, unmodified `y` colour.

Global constraints are always terminal nodes without any outgoing nodes; thus, as a consequence, updating them only once, at the end of the standard WL steps, will result in the same output as updating them during the standard WL steps.

Finally, once the final histograms are extracted from the graphs, we need to normalise the features. This is a necessary step since the values in the histograms will grow proportionally to the size of the graph. Thus, if not normalised, the produced histograms will be comparable only with those produced by graphs of the same size, making it impossible to generalise.

We decided to normalise the colour histograms by dividing each value in the histogram by the total count of nodes. Similarly, the pairs histogram is normalised by the total pairs count. This normalisation step will generate a final feature set with values always smaller than or equal to 1.

4.6 Train and Test Feature Extraction Process

To train a ML model using WL-based features, we have to standardise the histogram to force them to share the same colours. Since all possible colours are not known a priori, we first need to record all colours seen in the training instances and then update the histograms of each training instance with a 0 coefficient for each colour not present in the histogram.

During testing, if we encounter a new colour not present in the training data, we simply discard it and ignore the update.

5 Experimental Results

To test our methodology, we have implemented the graph conversion algorithm on top of the Flatzinc modelling language. We have then taken the instances from three Minizinc challenges (2023, 2024, and 2025), for a total of 300 instances and designed three different algorithm selection tasks around it: (i) algorithm selection based on the Borda count score on three solvers where the objective is to maximise the score, (ii) the same algorithm selection task but where the objective is to maximise the prediction accuracy and (iii) running a solver in its sequential and parallel version, we seek to predict if the parallel solver will meaningfully increase the performance with respect to the sequential version. The evaluation metrics will be predictive accuracy.

Each task will be compared against a baseline and the fzn2feat features: a feature set specifically designed for algorithm selection tasks for constraint programming. Due to the shallow depth of our graphs, we decided to test WL-based features with one or two aggregation steps.

For each task, we trained three different ML models: (i) Support Vector Machine (SVM), (ii) Random Forest and (iii) a simple, three layer, multi-layer-perceptron (MLP). And the training process followed a four-steps pipeline:

1. Perform grid search on a predefined set of hyperparameter configurations and score each hyperparameter configuration based on the mean 5-fold cross-validation score.
2. Select the hyperparameter configuration with the best score. Ties are broken by selecting the hyperparameter configuration with the lowest chance of overfitting:
   - Lowest regularisation parameter, gamma and using the rbf kernel for SVM.
   - Smallest number of trees and depth for Random Forest.
   - Lowest number of training steps and smallest alpha for MLP.
3. Test the performance of the best configuration on a range of smaller features set obtained by applying PCA decomposition on the original features set. The tested values are in the range $[20, \min(\text{n\_features}, \text{n\_training-datapoints})[$.
4. Select the feature set with the highest score between those obtained by applying PCA decomposition and the original one.

5.1 Borda Count Score Maximisation

In this task, we compared three sequential solvers on the 300 instances. The solvers were run without forcing them to follow the search annotations on a very old CPU (fill in) for a total of 20 minutes. On each instance, solvers have been scored based on the same borda count scoring strategy followed by the Minizinc challenge.[2] The task has been modeled as a classification task where the prediction label corresponded to the solver with the highest score. Ties have been broken by assigning the label to the solver with the highest overall score. The three solver we selected are `Or-Tools`, `Chuffed` and

[2] https://www.minizinc.org/challenge/2025/

`Cplex`. From the final dataset, we removed one instance which was trivially unsat (and from which it was impossible to extract features) and all instances where no solver ever found a solution. In the end, the final dataset was composed of 289 instances. We performed 5-fold cross-validation using a stratify strategy repeating the split 10 times with different random seeds for a total of 50 test. The train-test split was performed using Scikit-learn StratifiedKFold function[3] where the stratification was performed on the gap between the virtual best score (the maximum theoretical score obtainable) and the worse possible score.

References

1 Léo Boisvert, Hélène Verhaeghe, and Quentin Cappart. Towards a generic representation of combinatorial problems for learning-based approaches. In *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, pages 99–108. Springer, 2024.

2 Quentin Cappart, Didier Chételat, Elias B Khalil, Andrea Lodi, Christopher Morris, and Petar Veličković. Combinatorial optimization and reasoning with graph neural networks. *Journal of Machine Learning Research*, 24(130):1–61, 2023.

3 Elias B Khalil, Christopher Morris, and Andrea Lodi. Mip-gnn: A data-driven framework for guiding combinatorial solvers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 10219–10227, 2022.

4 Sandra Kiefer. *Power and limits of the Weisfeiler-Leman algorithm*. PhD thesis, Dissertation, RWTH Aachen University, 2020, 2020.

5 Christopher Morris, Yaron Lipman, Haggai Maron, Bastian Rieck, Nils M Kriege, Martin Grohe, Matthias Fey, and Karsten Borgwardt. Weisfeiler and leman go machine learning: The story so far. *Journal of Machine Learning Research*, 24(333):1–59, 2023.

[3] https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html